UNIVERSIDAD MARIANO GÁLVEZ

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

Guion de la Presentación Final

Entregable final del curso

Proyecto	EDUVIAGT — Sistema Integral de Gestión de Becas
Curso	0910-037 · Análisis de Sistemas II · 8.º Ciclo
Catedrático / PO	Ing. Mario Fuentes
Equipo	EDUVIAGT — 6 integrantes
Versión	1.0
Fecha	2026-10-29

Guion de la Presentación Final

Proyecto: *EDUVIAGT — Sistema Integral de Gestión de Becas* ***Curso:*** *0910-037 · Análisis de Sistemas II · 8.º Ciclo* ***Duración estimada:*** *20–25 minutos + 5 minutos de preguntas* ***Base:*** *entregable «Presentación Final» del documento del proyecto final*

Este guion ordena la exposición final en **10 bloques**. Cada bloque indica el objetivo, el contenido mínimo, la evidencia disponible en el repositorio y la demostración que debe accompanyarlo. La regla principal: **cada afirmación técnica se apoya en un archivo del repositorio**; no se afirma nada que el código no respalde.

0. Preparación previa (30 minutos antes)

#	Verificación	Comando / acción
1	El proyecto compila	`npm run build`
2	La suite de pruebas está en verde	`npm run test:api` y `cd apps/web && npm run test:e2e`
3	La base de datos de demostración está sembrada	`npx prisma db seed` en `apps/api`
4	La aplicación está levantada	`npm run up`
5	Los datos de la demo están listos	1 convocatoria abierta con monto > Q10,000 (para la demo de 2.ª aprobación), 1 solicitud en `EVALUADA`, 1 sesión de comité con quórum
6	El repositorio está abierto con la rama `develop` y el historial visible	`git log --oneline --graph`
7	El diagrama de arquitectura está listo para pantalla	`docs/architecture/interactive/architecture.html`

1. Apertura (1 min)

Objetivo: presentar el equipo y el producto en una frase.

- Saludo al docente y a los stakeholders.
- Presentación del equipo: 6 integrantes, roles Scrum y técnicos.
- Frase de posicionamiento: *«EDUVIAGT sustituye el proceso manual de becas del MINEDUC por una plataforma con

trazabilidad de punta a punta»*.

Evidencia: [Acta de Constitución](01-acta-constitucion-equipo.md) §2.

2. El problema y su análisis (3 min)

Objetivo: demostrar que el equipo entendió el problema antes de construir.

Contenido mínimo:

1. Los seis problemas del enunciado (retrasos, duplicidad, pérdida de documentos, poca trazabilidad, dificultad estadística, falta de transparencia) y **cómo se manifiestan hoy**.

2. La **causa raíz** de cada uno: no es «falta de herramientas», es que el proceso no tiene **estado explícito** (las reglas viven en la cabeza de las personas) ni **registro** (no hay historial de las decisiones).

3. La consecuencia de negocio: sin expediente único no hay informe, sin historial no hay auditoría, sin automatismo no hay escala.

Evidencia: [Documento de Visión](03-documento-vision.md) §2–§3.

3. El producto: qué hace (3 min)

Objetivo: mostrar cobertura completa de los requisitos mínimos.

• Recorrer el ciclo de vida en 6 pasos: **Convocatoria → Postulación → Documentación → Evaluación → Comité → Resolución**.

• Marcar en pantalla dónde se cumple cada uno de los **9 requisitos mínimos** del enunciado con una tabla de cobertura (requisito → módulo → historia → endpoint).

• Destacar dos funcionalidades que van más allá del mínimo:

• **Segunda aprobación por monto** (> Q10,000) — separación de poderes.

• **Montos flexibles por tramos de ingreso familiar** — la política económica es un dato, no código.

Evidencia: [Documento de Diseño](07-documento-diseno.md) §2.2 · [Documento de Visión](03-documento-vision.md) §5.

Demostración (3 min):

1. Portal público: listado de convocatorias con filtros → detalle.

2. Login de postulante → dashboard → formulario multi-paso → carga de un documento → checklist → envío.

3. Consulta pública del estado por código.

4. Arquitectura: qué decidimos y por qué (4 min)

Objetivo: mostrar el razonamiento arquitectónico, no solo el resultado.

1. **El estilo elegido:** monolito modular por capas, en un monorepo con dos aplicaciones independientes.

2. **El razonamiento en 30 segundos:** el problema es un flujo lineal con reglas de estado complejas y fuerte necesidad de trazabilidad; **no** es un problema de escala o de distribución. Un monolito modular da bajo acoplamiento sin el costo operativo de los microservicios, y deja la puerta abierta a la extracción.

3. **La respuesta a los 6 criterios del enunciado** (mantenibilidad, escalabilidad, bajo acoplamiento, reutilización, facilidad de cambio, complejidad) — dar una línea por criterio, sin leer la tabla.

4. **El argumento decisivo:** la regla que más cambia (transiciones) está **expresada como datos**, y los requisitos nuevos del Sprint 8 se incorporaron sin reescribir el dominio.

5. **Los trade-offs asumidos**, con honestidad: sin Repository, sin bus de eventos, sin caché, rate limiting en

memoria. Se enuncian como deuda técnica declarada, no como decisiones sublimes.

Evidencia: [Justificación de la Arquitectura](05-justificacion-arquitectura.md) §3 y §6 · [Documento de Arquitectura](04-documento-arquitectura.md).

Soporte visual: diagrama de componentes y diagrama de clases del dominio ([Documento de Diseño](07-documento-diseno.md) §4).

5. Patrones de diseño (4 min)

Objetivo: Demostrar que los patrones resuelven problemas reales, y no que se copiaron de un catálogo.

Formato para cada patrón (máximo 40 segundos): problema → por qué ese patrón → qué alternativa se descartó.

#	Patrón	Frase clave
1	**State**	La FSM es una **tabla de transiciones**: añadir el estado de segunda aprobación fue editar datos, no reescribir lógica.
2	**Chain of Responsibility**	Excepción individual → permiso del rol → **denegar**. Con el `DefaultDenyHandler` como último eslabón, el sistema **falla cerrado**.
3	**Adapter**	El dominio no sabe si el fichero vive en disco o en S3; migrar es cambiar un binding de token.
4	**Proxy**	El asistente nunca deja al usuario sin respuesta: si el LLM falla o expira, degrada a la base de conocimiento local.
5	**Strategy**	El sistema funciona **con y sin IA externa**: el CI es determinista porque siempre resuelve al fallback.
6	**Decorator**	`@Permisos(...)` deja la política de acceso **en una línea, junto al endpoint**, con un único punto de enforcement.
7	**Singleton / Factory DI**	Un solo pool de conexiones; los cuatro puntos de entrada al exterior se construyen en un único lugar.

Remate honesto (30 segundos): los patrones que **no** están implementados (Observer, Builder, Repository, CQRS) están documentados como tales, y los de uso parcial (Template Method, Command, Circuit Breaker) se declaran parciales con su limitación concreta.

Evidencia: [Justificación de Patrones](06-justificacion-patrones-de-diseno.md) — con las 4 preguntas por patrón.

6. SOLID y calidad del diseño (2 min)

Objetivo: mostrar disciplina de diseño, no solo cumplimiento.

- Una línea por principio con **un ejemplo real del repositorio**:
- **S**: `AuthService` / `EliminarCuentaService` / `MontosService` separados.
- **O**: añadir un proveedor de IA o un estado es extender una tabla o registrar un provider.
- **L**: `DocumentStorage` con `FilesystemStorageAdapter` sustituible.
- **I**: interfaces segregadas por dominio (`DocumentStorage`, `PdfRenderer`, `ProveedorIA`).
- **D**: los servicios dependen de interfaces, no de implementaciones concretas.
- La evidencia de que el diseño aguanta el cambio: **5 requisitos nuevos en el Sprint 8 sin reescribir el dominio**.

Evidencia: [`docs/architecture/solid-principles.md`](../architecture/solid-principles.md).

7. Proceso: Scrum, Git y calidad (3 min)

Objetivo: mostrar que el equipo trabajó con disciplina, no solo que sabe programar.

1. **Scrum:** 9 sprints, cada uno con `goal.md`, `backlog.md`, `daily-log.md`, `review.md` y `retrospective.md`.

Enseñar el árbol de `docs/scrum/` — la estructura habla por sí misma.

2. **Git:** mostrar el historial (`git log --oneline --graph`) con Conventional Commits, ramas `feature/*`,

`develop` como rama de integración y `master` protegida. Enseñar **un PR con revisión real** (con los comentarios ■■■).

3. **Calidad:** 26 suites unitarias + 8 E2E con axe, umbral de cobertura, 6 jobs de CI, code review obligatorio

y DoD con 10 puntos.

4. **Documentación:** `docs/` con índice maestro, 22 ADR, diagramas Mermaid y los 15 entregables del curso.

5. **Deuda técnica declarada:** mencionar que el equipo **documenta lo que no está resuelto** y por qué.

Evidencia: [`docs/scrum/`](../scrum/) · [`docs/standards/git-conventions.md`](../standards/git-conventions.md) · [Plan de Calidad](08-plan-aseguramiento-calidad.md) · [Plan de Pruebas](09-plan-pruebas-agil.md).

8. Adaptación al cambio (2 min)

Objetivo: demostrar la capacidad que el enunciado evalúa explícitamente («adaptarse al cambio»).

Presentar la tabla de cambios recibidos y respuesta del equipo (Sprint 8):

Cambio	Respuesta	Impacto arquitectónico
Segunda aprobación > Q10,000	Nuevo estado en la FSM + `SolicitudAprobacion` + validación en servidor	**Ninguno**: se extender la tabla de transiciones y añadir un eslabón
Montos por estrato socioeconómico	`ConvocatoriaReglaMonto` configurable	**Ninguno**: la política es un dato
Derecho al olvido	Anonimización conservando la trazabilidad	**Ninguno**: servicio nuevo, dominio intacto
Exigencia de auditoría	Bitácora *append-only* + auditoría reforzada	**Ninguno**: transversal
Accesibilidad WCAG 2.2 AA	Foco, skip link, contraste, `prefers-reduced-motion`	**Ninguno**: transversal de frontend

Frase de cierre del bloque: *«El arquitectura que elegimos es lo que hizo que ninguno de estos cambios requiriera un rediseño.»*

Evidencia: [Documento de Visión](03-documento-vision.md) §7 · ADR-012 a ADR-016.

9. Estado actual y cierre (2 min)

Objetivo: cerrar con honestidad y con lo que queda.

1. **Lo que funciona:** los 9 requisitos del enunciado, más segunda aprobación, montos flexibles, bitácora,

anonimización, accesibilidad, asistente y CI/CD.

2. **Lo que no está terminado:** la tabla de deuda técnica (§14 del Manual Técnico) leída en voz alta y con

su mitigación.

3. **Lo que vendría después:** pagos y conciliación bancaria, formalización de contratos, renovación de becas,

lista de espera, 2FA, almacenamiento en la nube, RAG con embeddings, app móvil — el backlog futuro ya está priorizado.

4. **Aprendizajes del equipo:** qué aprendieron sobre Scrum, arquitectura y control de versiones que no sabían al

empezar.

10. Preguntas y cierre (5 min)

- Agradecer al docente y a los stakeholders.
- Dejar los enlaces al repositorio, a la documentación y a la demostración desplegada.

Preguntas previsibles del docente y respuesta prepared:

Pregunta probable	Respuesta
«¿Por qué no usaron microservicios?»	El volumen y el tamaño del equipo no lo justifican; el monolito modular deja la puerta abierta a la extracción. Ver [Justificación de la Arquitectura](05-justificacion-arquitectura.md) §3.6

Pregunta probable	Respuesta
«¿Cómo garantizan que nadie aprobé su propia beca?»	La validación es **en el servidor** (`AuthzService.assertSegundaAprobacion`), no en la interfaz: la segunda firma debe ser de otra persona. Ver [Documento de Diseño](07-documento-diseno.md) UC-31
«¿Qué pasa si el LLM externo se cae?»	El Proxy degrada automáticamente al modo de base de conocimiento local; el usuario siempre recibe respuesta. Ver [Justificación de Patrones](06-justificacion-patrones-de-diseno.md) §3.4
«¿Su cobertura de pruebas es suficiente?»	Está por encima del umbral definido por el equipo, pero reconocemos que 49 % en *statements* deja bordes sin cubrir; la mejora está priorizada en el Sprint 9. Ver [Plan de Pruebas](09-plan-pruebas-agil.md)
«¿Cómo manejan los datos personales?»	Minimización, validación estricta de ficheros, anonimización en la eliminación de cuenta y bitácora *append-only*. Ver [Manual Técnico](10-manual-tecnico.md) §12
«¿Qué patrón les costó más?»	El más discutido fue State: la primera versión usaba condicionales y se volvió inmanejable; la tabla de transiciones fue la respuesta. Ver [Justificación de Patrones](06-justificacion-patrones-de-diseno.md) §3.6

Material de apoyo

Elemento	Ubicación
Diagrama interactivo de arquitectura	`docs/architecture/interactive/architecture.html`
Diagramas Mermaid	`docs/architecture/diagrams.md`
Demo funcional desplegada	entorno de staging (job `deploy-staging`)
Datos de demostración	`docs/architecture/datos-demostracion.md`
Manuales de usuario	`docs/manual-usuario/`
Índice de toda la documentación	`docs/README.md`
Índice de los entregables	[`docs/entregables/README.md`](README.md)

Última actualización: 2026-10-28